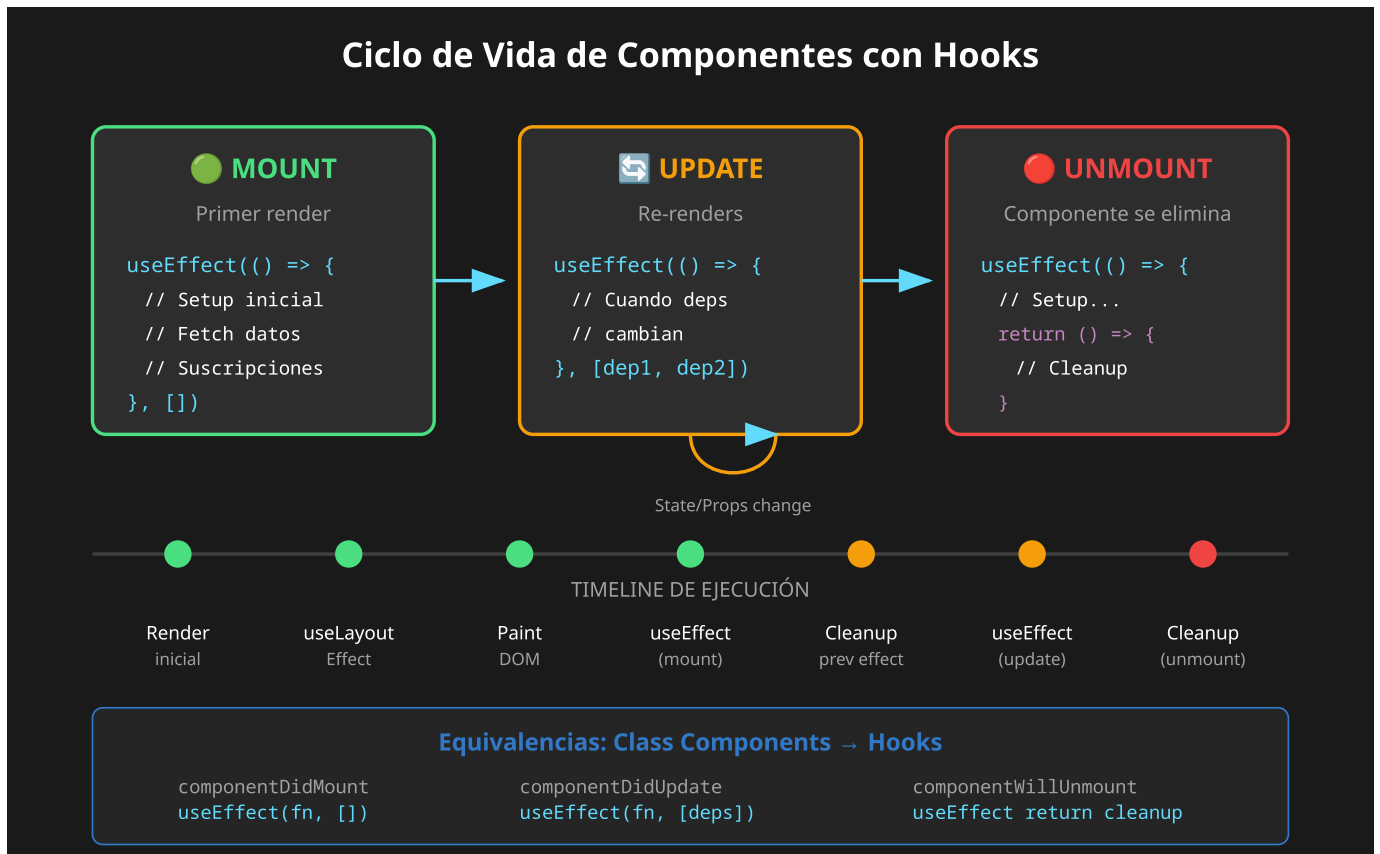


Ciclo de Vida de Componentes con Hooks

Objetivos

- Entender el ciclo de vida de componentes funcionales
- Mapear métodos de clases a hooks equivalentes
- Dominar useEffect para cada fase del ciclo de vida
- Evitar errores comunes con efectos

1. Ciclo de Vida en React



Componentes de Clase (Legado)

```
// ❌ FORMA ANTIGUA - Solo referencia  
class MyComponent extends React.Component {  
  componentDidMount() {  
    // Se ejecuta DESPUÉS del primer render  
  }  
  
  componentDidUpdate(prevProps, prevState) {  
    // Se ejecuta DESPUÉS de cada actualización  
  }  
  
  componentWillUnmount() {  
    // Se ejecuta ANTES de desmontar  
  }  
}
```

```
}  
}
```

Componentes Funcionales (Moderno)

```
// ✅ FORMA MODERNA - Con hooks  
const MyComponent: React.FC = () => {  
  // 1. MOUNT: Después del primer render  
  useEffect(() => {  
    console.log('Componente montado');  
  }, []); // Array vacío = solo en mount  
  
  // 2. UPDATE: Después de cada render  
  useEffect(() => {  
    console.log('Componente renderizado');  
  }); // Sin dependencias = cada render  
  
  // 3. UNMOUNT: Antes de desmontar  
  useEffect(() => {  
    return () => {  
      console.log('Componente se desmontará');  
    };  
  }, []);  
  
  return <div>Mi Componente</div>;  
};
```

2. Fases del Ciclo de Vida

Fase 1: Montaje (Mount)

```
// =====  
// QUÉ: useEffect con array vacío para fase de montaje  
// PARA: Ejecutar código una sola vez al montar el componente  
// IMPACTO: Equivalente a componentDidMount en clases  
// =====  
  
const MountExample: React.FC = () => {  
  const [data, setData] = useState<string | null>(null);  
  
  // componentDidMount equivalente  
  useEffect(() => {  
    console.log('✅ Componente montado');  
  
    // Típico: fetch inicial de datos  
    const fetchData = async () => {  
      const response = await fetch('/api/data');  
      const result = await response.json();  
      setData(result);  
    };  
  
    fetchData();  
  }, []);  
};
```

```

// Típico: suscripciones
const subscription = someService.subscribe();

// Cleanup para cuando se desmonte
return () => {
  subscription.unsubscribe();
};
}, [])); // ← Array vacío = solo mount

return <div>{data}</div>;
};

```

| Tiempo de inicio | < 1 segundo | 10-30 segundos | | HMR | Instantáneo | 1-5 segundos | | Configuración | vite.config.ts | Eject necesario | | Build tool | Rollup (ESBuild) | Webpack | | Tamaño bundle | Más pequeño | Más grande |

Fase 2: Actualización (Update)

```

// =====
// QUÉ: useEffect con dependencias para fase de actualización
// PARA: Re-ejecutar lógica cuando cambian valores específicos
// IMPACTO: Control granular sobre cuándo se ejecutan efectos
// =====

interface UserProfileProps {
  userId: number;
}

const UserProfile: React.FC<UserProfileProps> = ({ userId }) => {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  // componentDidUpdate equivalente (para userId)
  useEffect(() => {
    console.log('🔄 userId cambió, re-fetching...');

    let isCancelled = false;

    const fetchUser = async () => {
      setLoading(true);
      try {
        const response = await fetch(`/api/users/${userId}`);
        const data = await response.json();

        // Verificar que no se canceló
        if (!isCancelled) {
          setUser(data);
        }
      } finally {
        if (!isCancelled) {
          setLoading(false);
        }
      }
    }

    fetchUser();
  }, [userId]);
};

```

```

    // Cleanup: marcar como cancelado si userId cambia antes
    return () => {
      isCancelled = true;
    };
  }, [userId]); // ← Se ejecuta cuando userId cambia

  if (loading) return <p>Cargando...</p>;
  return <div>{user?.name}</div>;
};

```

Fase 3: Desmontaje (Unmount)

```

// =====
// QUÉ: Función de cleanup en useEffect para desmontaje
// PARA: Limpiar recursos (timers, suscripciones, listeners)
// IMPACTO: Previene memory leaks y comportamiento inesperado
// =====

const TimerComponent: React.FC = () => {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    console.log('🕒 Timer iniciado');

    const interval = setInterval(() => {
      setSeconds((prev) => prev + 1);
    }, 1000);

    // componentWillUnmount equivalente
    return () => {
      console.log('🛑 Timer detenido (cleanup)');
      clearInterval(interval);
    };
  }, []);

  return <p>Segundos: {seconds}</p>;
};

// Componente padre que monta/desmonta
const App: React.FC = () => {
  const [showTimer, setShowTimer] = useState(true);

  return (
    <div>
      <button onClick={() => setShowTimer((prev) => !prev)}>
        {showTimer ? 'Ocultar' : 'Mostrar'} Timer
      </button>

      {showTimer && <TimerComponent />}
      {/* Al ocultar, se ejecuta el cleanup del timer */}
    </div>
  );
};

```

3. Mapa de Equivalencias

Método de Clase	Hook Equivalente
constructor	useState (estado inicial)
componentDidMount	useEffect(() => {}, [])
componentDidUpdate	useEffect(() => {}, [deps])
componentWillUnmount	useEffect(() => { return () => {} }, [])
shouldComponentUpdate	React.memo, useMemo, useCallback
getDerivedStateFromProps	useState + lógica en render
getSnapshotBeforeUpdate	useLayoutEffect (casos especiales)

4. useEffect vs useLayoutEffect

```
// =====
// QUÉ: Diferencia entre useEffect y useLayoutEffect
// PARA: Elegir el hook correcto según el caso de uso
// IMPACTO: useLayoutEffect evita parpadeos visuales en DOM
// =====

import { useEffect, useLayoutEffect, useState, useRef } from 'react';

const LayoutEffectExample: React.FC = () => {
  const [value, setValue] = useState(0);
  const boxRef = useRef<HTMLDivElement>(null);

  // ❌ useEffect: Puede causar "flash" visual
  useEffect(() => {
    if (boxRef.current) {
      // Se ejecuta DESPUÉS de que el usuario ve el DOM
      boxRef.current.style.transform = `translateX(${value * 100}px)`;
    }
  }, [value]);

  // ✅ useLayoutEffect: Sin "flash", se ejecuta ANTES del paint
  useLayoutEffect(() => {
    if (boxRef.current) {
      // Se ejecuta ANTES de que el usuario vea cambios
      // Útil para medir DOM o hacer animaciones inmediatas
      const rect = boxRef.current.getBoundingClientRect();
      console.log('Posición:', rect.left);
    }
  }, [value]);

  return (
    <div>
      <div
        ref={boxRef}
        className="box">
        Box
      </div>
    </div>
  );
}
```

```

    </div>
    <button onClick={() => setValue((v) => v + 1)}>Mover</button>
  </div>
);
};

```

Cuándo usar cada uno

Caso de Uso	Hook Recomendado
Fetch de datos	useEffect
Suscripciones	useEffect
Timers	useEffect
Medir elementos DOM	useLayoutEffect
Sincronizar scroll	useLayoutEffect
Tooltips que dependen de posición	useLayoutEffect

5. Orden de Ejecución

```

// =====
// QUÉ: Demostración del orden de ejecución de efectos
// PARA: Entender cómo React procesa componentes padre/hijo
// IMPACTO: Debuggear problemas de timing en efectos
// =====

const Parent: React.FC = () => {
  console.log('1 Parent: render');

  useLayoutEffect(() => {
    console.log('4 Parent: useLayoutEffect');
    return () => console.log('Parent: cleanup layoutEffect');
  }, []);

  useEffect(() => {
    console.log('6 Parent: useEffect');
    return () => console.log('Parent: cleanup effect');
  }, []);

  return (
    <div>
      <Child />
    </div>
  );
};

const Child: React.FC = () => {
  console.log('2 Child: render');

  useLayoutEffect(() => {
    console.log('3 Child: useLayoutEffect');
    return () => console.log('Child: cleanup layoutEffect');
  }, []);
};

```

```

}, []);

useEffect(() => {
  console.log('5 Child: useEffect');
  return () => console.log('Child: cleanup effect');
}, []);

return <p>Child Component</p>;
};

// ORDEN DE CONSOLA:
// 1 Parent: render
// 2 Child: render
// 3 Child: useLayoutEffect (hijos primero)
// 4 Parent: useLayoutEffect
// 5 Child: useEffect (hijos primero)
// 6 Parent: useEffect

```

6. Patrones Comunes del Ciclo de Vida

Patrón: Efecto con Dependencias Múltiples

```

interface SearchParams {
  query: string;
  page: number;
  filters: string[];
}

const SearchResults: React.FC<SearchParams> = ({ query, page, filters }) => {
  const [results, setResults] = useState<Item[]>([]);
  const [loading, setLoading] = useState(false);

  // Se ejecuta cuando CUALQUIERA de las dependencias cambia
  useEffect(() => {
    const abortController = new AbortController();

    const search = async () => {
      setLoading(true);
      try {
        const params = new URLSearchParams({
          q: query,
          page: String(page),
          filters: filters.join(','),
        });

        const response = await fetch(`/api/search?${params}`, {
          signal: abortController.signal,
        });

        const data = await response.json();
        setResults(data);
      } catch (error) {
        if (error instanceof Error && error.name !== 'AbortError') {
          console.error('Error en búsqueda:', error);
        }
      }
    };
  }, [query, page, filters]);
};

```

```

    }
  } finally {
    setLoading(false);
  }
};

search();

return () => {
  abortController.abort();
};
}, [query, page, filters]); // Todas las dependencias

return (
  <div>
    {loading ? (
      <p>Buscando...</p>
    ) : (
      <ul>
        {results.map((item) => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    )}
  </div>
);
};

```

Patrón: Efecto Condicional

```

const ConditionalEffect: React.FC<{ enabled: boolean; userId: number }> = ({
  enabled,
  userId,
}) => {
  const [data, setData] = useState(null);

  useEffect(() => {
    // No ejecutar si está deshabilitado
    if (!enabled) {
      return;
    }

    const fetchData = async () => {
      const response = await fetch(`/api/users/${userId}`);
      setData(await response.json());
    };

    fetchData();
  }, [enabled, userId]); // enabled como dependencia

  return <div>{data ? JSON.stringify(data) : 'Sin datos'}</div>;
};

```

Patrón: Sincronizar con Prop


```
// Cuando necesitas "resetear" estado basado en prop
const EditableField: React.FC<{ initialValue: string; id: string }> = ({
  initialValue,
  id,
}) => {
  const [value, setValue] = useState(initialValue);

  // Sincronizar cuando cambia el ID (nuevo item)
  useEffect(() => {
    setValue(initialValue);
  }, [id]); // Solo cuando cambia id, no initialValue

  return (
    <input
      value={value}
      onChange={(e) => setValue(e.target.value)}
    />
  );
};
```

7. Errores Comunes y Soluciones

Error 1: Dependencias Faltantes

```
// ❌ MAL: Falta dependencia
const BadComponent: React.FC<{ userId: number }> = ({ userId }) => {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then((r) => r.json())
      .then(setUser);
  }, []); // ⚠ Warning: falta userId en dependencias

  return <div>{user?.name}</div>;
};

// ✅ BIEN: Incluir todas las dependencias
const GoodComponent: React.FC<{ userId: number }> = ({ userId }) => {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then((r) => r.json())
      .then(setUser);
  }, [userId]); // ✅ Correcto

  return <div>{user?.name}</div>;
};
```

Error 2: Loop Infinito

```
// ❌ MAL: Objeto como dependencia causa loops
const LoopComponent: React.FC = () => {
  const [data, setData] = useState([]);
  const options = { limit: 10 }; // Nuevo objeto cada render

  useEffect(() => {
    fetch(`/api/items?limit=${options.limit}`)
      .then((r) => r.json())
      .then(setData);
  }, [options]); // ⚠️ Loop infinito: options siempre es nuevo

  return (
    <ul>
      {data.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
};

// ✅ BIEN: Usar useMemo o valores primitivos
const FixedComponent: React.FC = () => {
  const [data, setData] = useState([]);
  const limit = 10; // Primitivo, no cambia

  // O usar useMemo para objetos
  const options = useMemo(() => ({ limit: 10 }), []);

  useEffect(() => {
    fetch(`/api/items?limit=${limit}`)
      .then((r) => r.json())
      .then(setData);
  }, [limit]); // ✅ Sin loop

  return (
    <ul>
      {data.map((item) => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
};
```

Error 3: Estado después de Unmount

```
// ❌ MAL: Puede causar memory leak
const LeakyComponent: React.FC = () => {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch('/api/slow-data')
      .then((r) => r.json())
      .then(setData); // ⚠️ Si se desmonta antes, error
  }, []);
```

```

    return <div>{data}</div>;
  };

// ✅ BIEN: Verificar si sigue montado
const SafeComponent: React.FC = () => {
  const [data, setData] = useState(null);

  useEffect(() => {
    let isMounted = true;

    fetch('/api/slow-data')
      .then(r => r.json())
      .then(result => {
        if (isMounted) {
          setData(result);
        }
      });

    return () => {
      isMounted = false;
    };
  }, []);

  return <div>{data}</div>;
};

```

8. Hook useLifecycle (Custom)

```

// =====
// QUÉ: Hook useLifecycle para debugging del ciclo de vida
// PARA: Visualizar mount/update/unmount durante desarrollo
// IMPACTO: Facilita entender cuándo se ejecuta cada fase
// =====

import { useEffect, useRef } from 'react';

interface LifecycleCallbacks {
  onMount?: () => void;
  onUpdate?: () => void;
  onUnmount?: () => void;
}

const useLifecycle = (
  componentName: string,
  callbacks?: LifecycleCallbacks
): void => {
  const renderCount = useRef(0);
  const isFirstRender = useRef(true);

  // Mount
  useEffect(() => {
    console.log(`[${componentName}] 🟢 Montado`);
    callbacks?.onMount?.();
  }, []);

```

```

return () => {
  console.log(`[${componentName}] 🛑 Desmontado`);
  callbacks?.onUnmount?.();
};
}, [componentName, callbacks]);

// Update
useEffect(() => {
  renderCount.current += 1;

  if (isFirstRender.current) {
    isFirstRender.current = false;
    return;
  }

  console.log(`[${componentName}] 🔄 Actualizado (render #${renderCount.current})`);
  callbacks?.onUpdate?.();
});
};

export { useLifecycle };

// Uso
const DebugComponent: React.FC = () => {
  useLifecycle('DebugComponent', {
    onMount: () => console.log('Callback de mount'),
    onUpdate: () => console.log('Callback de update'),
    onUnmount: () => console.log('Callback de unmount')
  });

  return <div>Debug</div>;
};

```

Recursos Adicionales

- [React Docs: useEffect](#)
- [React Docs: useLayoutEffect](#)
- [Dan Abramov: A Complete Guide to useEffect](#)

✅ Checklist de Comprensión

- ☐ Conozco las 3 fases del ciclo de vida
- ☐ Sé mapear métodos de clase a hooks
- ☐ Entiendo la diferencia entre useEffect y useLayoutEffect
- ☐ Conozco el orden de ejecución de efectos
- ☐ Sé evitar loops infinitos y memory leaks

Siguiente: [04-vite-configuracion.md](#)